FIAP

Faculdade de Informática e Administração Paulista

Computational Thinking Using Phyton

Sprint 1

INTEGRANTES

RM (SOMENTE NÚMEROS)	NOME COMPLEMENTO (SEM ABREVIAR)
554874	João Gabriel Boaventura Marques e Silva
558791	Lucas de Melo Pinheiro Pinho
551124	Lucas Leal das Chagas

SUMÁRIO

DESCRITIVO.....5 - 6

ARRAYS E MENU 7

CASO DIGITAR 1(CADASTRO) 8

CASO DIGITAR 2 (PRÉ-DIAGNÓSTICO).....9

CASO DIGITAR 3 (CHAMAR GUINCHO).....10

Caso digitar 4.....11

Caso digitar 5 11

Caso digitar 6 12

CASO DIGITAR 7 (LOGOFF) 13

Novidades. ..... 14 a 18

Novidades e vídeo. ..... 19 a 20

Descritivo

O objetivo principal do projeto Diagnoscar é fornecer um sistema de **pré-diagnóstico automotivo** que, por meio de uma interface interativa, permita aos usuários identificarem problemas comuns com seus veículos. Este projeto visa oferecer aos proprietários de veículos uma ferramenta acessível e fácil de usar que os ajude a economizar tempo e recursos, evitando visitas desnecessárias às oficinas mecânicas.

O projeto implementará os seguintes recursos:

1. **Cadastro de Clientes**: Os usuários podem criar uma conta fornecendo suas informações pessoais, incluindo detalhes do veículo.

2. **Pré-diagnóstico**: Com base nos relatos do usuário, o sistema identificará problemas potenciais usando um conjunto de palavras-chave relacionadas a problemas comuns.

3. **Encaminhamento para Oficina**: Se for identificado um problema que necessite de reparo imediato, o sistema oferecerá a opção de encaminhar o veículo para uma oficina credenciada, com a possibilidade de chamar um guincho se necessário.

4. **Suporte 24 horas por dia**: O sistema fornecerá números de contato para atendimento ao cliente disponível 24 horas por dia.

5. **Interface Intuitiva**: Os usuários poderão acessar todas as funcionalidades sem problemas, pois o menu do sistema será claro e fácil de usar.

6. **Base de Dados de Problemas**: Um dicionário de problemas e palavras-chave relevantes ajudará a encontrar rapidamente os problemas relatados pelos usuários.

7. **Histórico de Diagnósticos**: Para uso futuro, o sistema registrará todos os diagnósticos realizados, permitindo um acompanhamento contínuo da saúde do veículo.

Ao fornecer uma solução integrada para o diagnóstico de problemas veiculares, melhorar a experiência do usuário e aumentar a eficiência do serviço de manutenção automotiva, essas funcionalidades refletem o contexto do projeto Diagnoscar.

Arrays e Menu

Arrays de cadastro, problemas e relatos

```
cad_cliente = {} #array para cadastro do cliente
pessoafisica = [] #array para cadastro do cliente


problems_chaves = {
    "vazamento de óleo": ["vazamento", "óleo", "mancha", "escorrendo"],
    "motor superaqueceu": ["superaquecimento", "motor quente", "temperatura alta"],
    "freio está com defeito": ["freio", "pedal baixo", "barulho", "falha de frenagem"],
    "motor cheira a queimado": ["cheiro queimado", "fumaça", "odor estranho", "fogo"]
}#array/dicionario para cadastro de problemas
rELATADO = []#array para cadastro de problemas relacionados
```

Loop para o Menu

```
#loop menu
while True:
    print("Bem vindo ao Diagnoscar!")
    print("Escolha uma opção:")
    print("1 - Criar conta")
    print("2 - Iniciar diagnóstico")
    print("3 - Encaminhar para oficina")
    print("4 - Sair")
    opcao = input("Opção: ")

```

Caso Digitar 1(Cadastro)

Variaveis que irão salvar as informações dadas pelo usuário para o cadastro

```python
# Parte de Cadastro
    if opcao == '1':
        nome = input("Digite o nome: ")

        cpf = input("Digite o CPF (sem espaços, ponto ou traço): ")
        if cpf in cad_cliente:
            print("CPF já cadastrado. Por favor, escolha outra opção.")
            continue
        if not cpf.isdigit() or len(cpf) != 11:
            print("CPF inválido. Deve conter 11 dígitos numéricos.")
            continue

        cnh = input("Digite a CNH: ")
        if len(cnh) !=11:
            print("CNH inválida. Deve conter 11 dígitos numéricos.")
            continue

        telefone = input("Digite o telefone: ")

        email = input("Digite o email: ")
        if "@" not in email or "." not in email:
            print("Email inválido. Digite um email válido.")
            continue

        endereco = input("Digite o endereço(Bairro, Rua e Nº(caso não houver coloque 00)): ")

        automoveis = input("Digite a marca e o modelos do(s) automóvel(s) (caso houver mais de 1, separade com vírgula): ").split(',')


        cad_cliente[cpf] = {'nome': nome, 'cnh': cnh, 'telefone': telefone, 'email': email, 'endereco': endereco, 'automoveis': automoveis}
        pessoafisica.append(cpf)
        print("Conta criada com sucesso.")
        print("==================")
        print("              ")
```

Caso digitar 2 (Pré-Diagnóstico)

Irá ser pedido o CPF do usuário, caso ele esteja na array “cad_cliente” será iniciado o pré-dignóstico, caso não seja encontrado, voltará ao menu. Após inicializado o pré-diagnostico, o código irá pedir qual carro, que o usuário relatou ter, está com problema e pedirá para o usuário relatar qual o problema, se o problema relato estiver em “problemas_chaves”, o código pedirá para o usuário digitar 3 para chamar um guincho. Se não ele irá orientar para ligar para ligar para Porto Seguro para ter um diagnóstico melhor

```
# Parte de Pré-Diagnóstico
    elif opcao == '2':
        cpf = input("Digite o CPF do cliente para começar o diagnóstico: ")
        if cpf in cad_cliente:
            print("Diagnóstico iniciado. Escolha o carro com problema:")
            for i, carro in enumerate(cad_cliente[cpf]['automoveis'], 1):
                print(f"{i} - {carro}")

            carro_selecionado = int(input("Digite o número correspondente ao carro com problema: "))
            print("             ")
            relato = input("Por favor, conte-me sobre o problema: ")
            problemas_encontrados = []

            for problema, palavras_chave in problems_chaves.items():
                for palavra_chave in palavras_chave:
                    if palavra_chave in relato.lower():
                        problemas_encontrados.append(problema)
                        break

            if problemas_encontrados:
                print("Problemas identificados:")
                for problema in problemas_encontrados:
                    print("-", problema)
                rELATADO.append((cpf, relato, problemas_encontrados))
                print("             ")
                print("Por favor digite 3, um guincho irá ser chamado para levar seu carro até uma oficina Porto Seguro.")
            else:
                print("Não conseguimos identificar o problema. Ligue para Porto Seguro para melhor diagnóstico.")
        else:
            print("Cliente não encontrado.")
```

Caso digitar 3 (Chamar Guincho)

Irá ser pedido o CPF do usuário, caso ele esteja na array “cad_cliente”, será exibido uma mensagem que o guincho chegará até o “endereço” que o usuário digitou no cadastro, caso precise de mais informações, ligar para o sac. Se caso o CPF não for encontrado, ele voltará ao menu.

```python
# Parte de Chamar Guincho
    elif opcao == '3':
        cpf = input("Digite o CPF do cliente para encaminhar à oficina: ")
        if cpf in cad_cliente:
            endereco_cliente = cad_cliente[cpf]['endereco']
            print(f"Estamos enviando um guinhco ao endereço: {endereco_client
e}")
            print(f"Ele irá deixar seu carro em uma oficina Credenciada da Po
rto Seguro")
            print("Para mais informações entre em contato")
            print("0800 727 2766 - Atendimento 24h.")
            print("0800 727 8736- Atendimento exclusivo para pessoas com defi
ciência auditiva. Disponível 24 horas de domingo a domingo")
            exit()

        else:
            print("Cliente não encontrado, Tente novamente.")
```

Caso digitar 4

Mostra contas cadastradas pelo usuário

```
# clientes cadastrados
def visualizar_clientes_cadastrados(cad_cliente):
    if cad_cliente:
        print("Clientes cadastrados:")
        for cliente in cad_cliente.values():
            print("CPF:", cliente.cpf)
            print("Nome:", cliente.nome)
            print("Endereço:", cliente.endereco)
            print("-------------------")
    else:
        print("==============================================================")
        print("Não há clientes cadastrados.")
```

Caso digitar 5

Mostra diagnósticos realizados

```
# diagnósticos realizados
def visualizar_diagnosticos_realizados(diagnosticos):
    if diagnosticos:
        print("==============================================================")
        print("Diagnósticos realizados:")
        for diagnostico in diagnosticos:
            print("CPF:", diagnostico.cpf)
            print("Relato do problema:", diagnostico.relato)
            print("Problemas identificados:")
            for problema in diagnostico.problemas:
                print("-", problema)
            print("-------------------")
    else:
        print("==============================================================")
        print("Não há diagnósticos realizados.")
```

Casa digitar 6

Sugestões de prevenções a se tomar baseado nos sistemas mais frequentes que tem problemas

```python
# Função para prever problemas futuros
def prever_problemas_futuros(diagnosticos):
    print("============================================================
    print("Previsão de Problemas Futuros")
    frequencia_problemas = {}

    for diagnostico in diagnosticos:
        for problema in diagnostico.problemas:
            frequencia_problemas[problema] = frequencia_problemas.get
            (problema, 0) + 1

    problemas_mais_frequentes = sorted(
        frequencia_problemas.items(), key=lambda x: x[1], reverse=True
    )

    if problemas_mais_frequentes:
        print("Problemas mais frequentes e suas contagens:")
        for problema, contagem in problemas_mais_frequentes:
            print(f"- {problema}: {contagem} vezes")
    else:
        print("Não há dados suficientes para prever problemas futuros.")
```

Caso digitar 7 (Logoff)

Caso Digitar 7: Irá acontecer o logoff

```
# Parte do Logoff
    elif opcao == '4':
        print("Saindo do sistema...")
        exit()
    else:
        print("Opção inválida. Tente novamente.")
```

Novidades 1.1

Relatório de Evoluções do Código: Sprint 1 para Sprint 2

Introdução

Este relatório descreve as principais evoluções feitas no código do projeto DiagnostiCar entre a Sprint 1 e a Sprint 2. O objetivo das mudanças foi melhorar a robustez, a legibilidade e a funcionalidade do sistema, que visa cadastrar clientes, diagnosticar problemas em automóveis e encaminhar clientes para oficinas.

Principais Evoluções

1. Validação de Entradas

Uma melhoria significativa entre as sprints foi a implementação de funções de validação para diversas entradas de dados. Na Sprint 1, o sistema não validava corretamente o CPF, telefone e email dos clientes. Na Sprint 2, foram adicionadas as seguintes funções de validação:

- `validar_cpf(cpf)`: Valida o CPF com base no formato e nos dígitos verificadores.
- `validar_telefone(telefone)`: Verifica se o telefone possui 10 ou 11 dígitos numéricos.
- `validar_email(email)`: Verifica se o email segue o formato padrão (exemplo@dominio.com).
- `validar_opcao_menu(opcao, max_opcao)`: Valida se a opção do menu está dentro do intervalo permitido.
- `entrada_nao_vazia(entrada)`: Verifica se uma entrada de texto não está vazia.

Essas validações aumentam a confiabilidade dos dados inseridos no sistema, evitando erros comuns de digitação e garantindo consistência.

2. Encapsulamento de Dados

Na Sprint 2, os dados dos clientes e diagnósticos foram encapsulados em classes, promovendo uma estrutura de dados mais organizada e orientada a objetos. Foram criadas duas classes principais:

- `Cliente`: Para armazenar os dados de cada cliente.

- `Diagnostico`: Para armazenar os dados de cada diagnóstico realizado.

Essa mudança facilita a manipulação e acesso aos dados, além de melhorar a manutenção do código.

3. Previsão de Problemas Futuros

Foi introduzida uma nova funcionalidade para prever problemas futuros com base nos diagnósticos realizados. A função `prever_problemas_futuros(diagnosticos)` analisa a frequência dos problemas encontrados e sugere ações preventivas. Essa funcionalidade acrescenta valor ao sistema, permitindo uma abordagem proativa na manutenção dos veículos.

4. Melhorias no Fluxo de Cadastro e Diagnóstico

O fluxo de cadastro de clientes e diagnóstico foi refinado para incluir validações de entrada mais robustas e interações mais intuitivas. Durante o cadastro de clientes, por exemplo, o sistema agora verifica se todos os campos obrigatórios foram preenchidos corretamente e se os formatos estão adequados.

5. Melhoria na Interface do Usuário

A interface do usuário foi aprimorada para tornar as interações mais claras e informativas. Foram adicionadas mensagens de erro detalhadas e feedbacks de sucesso para orientar o usuário durante o uso do sistema.

Comparativo de Código

Cadastro de Clientes - Antes (Sprint 1)

```python
def cadastrar_cliente():
    global cad_cliente # Declarando cad_cliente como global
    nome = input("Digite o nome: ")
    cpf = input("Digite o CPF: ")
    cnh = input("Digite a CNH: ")
    telefone = input("Digite o telefone: ")
    email = input("Digite o email: ")
    endereco = input("Digite o endereço: ")
    automoveis = input("Digite os modelos dos automóveis (separados por vírgula): ").split(',')

    cad_cliente[cpf] = {'nome': nome, 'cnh': cnh, 'telefone': telefone, 'email': email, 'endereco': endereco, 'automoveis': automoveis}
    pessoafisica.append(cpf)

print("==========================================================")
    print(f"Conta criada com sucesso para o cliente {nome}.")
```

Cadastro de Clientes - Depois (Sprint 2)

```python
def cadastrar_cliente(cad_cliente):
    nome = input("Digite o nome: ")
    while not entrada_nao_vazia(nome):
        print("Nome não pode estar vazio.")
        nome = input("Digite o nome: ")

    cpf = input("Digite o CPF: ")
    while not validar_cpf(cpf):
        print("CPF inválido. Deve conter 11 dígitos numéricos e ser válido.")
        cpf = input("Digite o CPF: ")

    cnh = input("Digite a CNH: ")

    telefone = input("Digite o telefone: ")
    while not validar_telefone(telefone):
        print("Telefone inválido. Deve conter 10 ou 11 dígitos numéricos.")
        telefone = input("Digite o telefone: ")

    email = input("Digite o email: ")
    while not validar_email(email):
        print("Email inválido. Formato esperado: exemplo@dominio.com")
        email = input("Digite o email: ")

    endereco = input("Digite o endereço: ")
    while not entrada_nao_vazia(endereco):
```

```
        print("Endereço não pode estar vazio.")
        endereco = input("Digite o endereço: ")

    automoveis = input("Digite os modelos dos automóveis (separados por vírgula): ").split(",")
    while not automoveis or any(not entrada_nao_vazia(carro) for carro in automoveis):
        print("Modelos dos automóveis não podem estar vazios.")
        automoveis = input("Digite os modelos dos automóveis (separados por vírgula): ").split(",")

    cliente = Cliente(nome, cpf, cnh, telefone, email, endereco, automoveis)
    cad_cliente[cpf] = cliente

print("==========================================================")
    print(f"Conta criada com sucesso para o cliente {nome}.")
```

Conclusão

As evoluções do código entre a Sprint 1 e a Sprint 2 resultaram em um sistema mais robusto, organizado e funcional. As principais melhorias incluíram a implementação de validações de entrada, encapsulamento de dados, previsão de problemas futuros e uma interface de usuário mais amigável. Essas mudanças tornam o sistema DiagnostiCar mais confiável e eficiente, melhorando a experiência do usuário e a qualidade dos diagnósticos realizados.

Novidades 1 . 2

Versão Original

1. Funções de Validação:

- As funções básicas para validar CPF, telefone e e-mail estavam implementadas corretamente, mas não haviam validações extensivas ou mensagens de erro detalhadas.

2. Cadastro e Alteração de Cliente:
   - A função de cadastro e alteração de cliente já realizava validações, mas não possuía algumas das verificações e tratamentos mais detalhados presentes na nova versão.
3. Diagnóstico:
   - A função de diagnóstico identificava problemas com base em palavras-chave, mas não havia uma seção dedicada para prever problemas futuros.
4. Exportação e API Externa:
   - A exportação para JSON estava presente, mas a consulta à API externa não estava implementada.
5. Menu:
   - O menu era funcional, mas não possuía algumas das melhorias de validação de entrada e feedback de erro encontradas na nova versão.

Versão Nova

1. Funções de Validação:
   - As funções de validação foram aprimoradas para fornecer mensagens de erro mais informativas e garantir que os dados inseridos sejam válidos. Adicionou-se validação para CPF, telefone e e-mail com mensagens claras para o usuário.
2. Cadastro e Alteração de Cliente:
   - Melhorias foram feitas para garantir que campos não fiquem vazios, e o código agora inclui mensagens informativas para o usuário. A alteração de dados do cliente agora permite atualização de campos específicos, mantendo valores padrão se o usuário não fornecer novos dados.

3. Diagnóstico e Previsão de Problemas:
    - A função prever_problemas_futuros foi adicionada, proporcionando insights sobre a frequência de problemas e sugestões de ações preventivas com base nesses dados.
4. Exportação e API Externa:
    - A função de exportação para JSON foi mantida, mas a nova versão agora inclui uma função para consultar uma API externa para validar CPFs, adicionando uma camada extra de verificação.
5. Menu:
    - O menu foi refinado para permitir uma melhor interação com o usuário. A validação das opções do menu e a verificação de entradas inválidas foram aprimoradas para garantir que o usuário possa navegar no sistema de maneira mais intuitiva.

Resumo das Evoluções

- Validações: Adicionadas e aprimoradas para fornecer mensagens de erro mais claras.
- Cadastro e Alteração: Melhorias na validação de campos e no fluxo de dados.
- Diagnóstico: Introduzida a função de previsão de problemas futuros.
- Exportação e API: Mantida a exportação para JSON e adicionada a consulta a uma API externa.
- Menu: Melhorias na navegação e validação de entradas.

https://youtu.be/7GyuiNgH4_4?si=d3p8bWF2FfTNbn_t